

EFREI Paris

Mastère Data Engineering et IA — Promotion 2025-26

Projet certifiant RNCP40875

Expert en Ingénierie de Données — Bloc 2 :

Pilotage et implémentation de solutions IA

Système Intelligent de Rétention Client & Estimation du Risque de Revenus

Un système multi-modèles ML/DL permettant de prédire le risque de churn client et d'aider les équipes CRM à prioriser les actions de rétention

Étudiant : Ervin GOBY, Rayan TOUMERT

Formation : Mastère Data Engineering et IA

Tutrice : Sarah MALAEB

Date : Avril 2026

Table des matières

1. Résumé.....	3
2. Introduction et Contexte.....	4
3. Analyse du besoin Utilisateur.....	4
4. Méthodologie de Travail et gestion de Projet	6
5. Référentiel de Données	8
6. Analyse Exploratoire des Données	10
7. Préparation et Transformation des Données	14
8. pipeline IA et Architecture.....	15
9. Implémentation Technique.....	17
10. évaluation Comparative des modèles.....	19
11. Interprétabilité et Analyse Métier des Résultats	21
12. Interface Utilisateur Dashboard Décisionnel	23
13. API REST	24
14. Résultats et Test de Démonstration	25
15. Gouvernance, responsabilité et Limites	26
16. Limites et Piste d'amélioration.....	28
17. Conclusion.....	29

1. Résumé

Ce projet répond à une problématique business concrète : comment une entreprise SaaS peut-elle anticiper les résiliations de ses clients et quantifier l'impact financier associé ? À partir d'un dataset de 10 000 clients, nous avons développé un pipeline Data Science complet combinant exploration des données, prétraitement, modélisation multi-algorithmes, interprétabilité et mise à disposition via un dashboard décisionnel et une API REST.

Le dataset utilisé est le Customer Churn Business Dataset issu de Kaggle, composé de 32 variables couvrant le profil client, les comportements d'usage, la facturation, le support et l'engagement. Le taux de churn observé est de 10,2%, ce qui représente un risque financier de 862 640 € de revenus potentiellement perdus.

Deux tâches prédictives ont été développées en parallèle : une classification binaire visant à prédire la variable **churn** (0 = le client reste, 1 = le client part), et une régression visant à estimer la variable **revenue_at_risk**, qui représente le revenu potentiellement perdu si le client résilie. Pour chacune, quatre algorithmes ont été entraînés et comparés dans des conditions strictement équivalentes. Le Random Forest s'est imposé pour la classification avec un ROC-AUC de **0,810** et un Recall de 0,853, permettant d'identifier 85 churners sur 100. Le MLP a obtenu les meilleures performances en régression avec une MAE de 136,89 €, ce qui signifie qu'en moyenne, notre estimation du revenu à risque ne se trompe que de 139 € par client un niveau de précision acceptable au regard de la complexité de la variable cible. Une analyse SHAP a ensuite été appliquée sur le Random Forest pour expliquer les prédictions : le score de satisfaction client (csat_score), le NPS et l'ancienneté ressortent comme les variables les plus déterminantes, tant au niveau global que pour l'explication individuelle de chaque client à risque.

La solution finale comprend un dashboard interactif développé avec Streamlit (4 pages : vue d'ensemble, analyse des données, comparaison des modèles, prédiction client en temps réel) et une API REST FastAPI avec 3 endpoints documentés (/health, /predict, /model-info). Les modèles sont sérialisés avec joblib et Keras et peuvent être déployés en production.

Technologies principales : Python, Pandas, Scikit-learn, TensorFlow/Keras, XGBoost, SHAP, Streamlit, FastAPI, Matplotlib, Seaborn, Plotly.

2. Introduction et Contexte

2.1 Problématique métier

Dans l'économie numérique actuelle, le modèle SaaS s'est imposé comme l'un des formats dominants de l'industrie logicielle. Son principe repose sur la récurrence : plutôt que de vendre un logiciel une fois, l'entreprise facture un abonnement mensuel ou annuel. Cette récurrence est à la fois sa force et sa fragilité : chaque résiliation, appelée churn, vient directement éroder la base de revenus, client après client.

Le churn n'est pas simplement une statistique. C'est un signal d'échec dont le coût est double : on perd le revenu futur du client partant et on doit en acquérir un nouveau pour compenser, un effort qui coûte entre 5 et 7 fois plus cher que de fidéliser un client existant.

Dans notre dataset de 10 000 clients, 1 021 ont churné, représentant 862 640€ de revenus perdus. La question n'est donc pas seulement "combien de clients allons-nous perdre ?" mais "lesquels, et combien cela coûtera-t-il ?" Notre système répond à cette double question via deux tâches prédictives : identifier les clients à risque (classification) et estimer le revenu à risque associé (régression).

2.2 Pourquoi le Machine Learning supervisé ?

Des règles métier manuelles seraient arbitraires et figées. Des indicateurs de reporting décrivent le passé sans anticiper l'avenir. Le Machine Learning supervisé apprend automatiquement les patterns qui précèdent un churn à partir des données historiques : profil, comportement, facturation, support, engagement et les applique pour scorer de nouveaux clients.

Cette approche s'adapte à la complexité des données, capture des relations non linéaires, fournit une probabilité plutôt qu'un signal binaire, et s'explique grâce à SHAP. Le même principe s'applique à la régression : la variable cible n'est plus binaire mais continue, il s'agit d'estimer le revenu à risque de chaque client.

3. Analyse du besoin Utilisateur

3.1 Utilisateur cible

La solution est principalement destinée aux équipes CRM, marketing et financières d'une entreprise SaaS. Chaque profil d'utilisateur a des besoins distincts que la solution doit être capable de satisfaire.

Le responsable CRM est l'utilisateur le plus opérationnel : il a besoin d'identifier rapidement les clients à contacter en priorité, avant qu'ils ne prennent la décision de résilier. Pour lui, chaque heure compte : un client qui a décidé de partir est beaucoup plus difficile à retenir qu'un client encore hésitant.

Le responsable marketing s'intéresse davantage à la dimension budgétaire : il souhaite mesurer l'impact des campagnes de rétention et allouer les ressources en fonction du risque financier réel de chaque segment. Savoir que les clients SME churnent à 30,9% contre 6% pour les particuliers lui permet par exemple de concentrer ses efforts sur ce segment prioritaire.

Le directeur financier a quant à lui une vision plus macro : il veut quantifier le risque de revenus en temps réel et anticiper l'impact sur les marges avant que les résiliations ne se concrétisent. Pour lui, l'estimation du revenu à risque (862 640 €) est l'indicateur le plus stratégique.

Enfin, le data analyst est l'utilisateur le plus technique du dashboard : il exploite les visualisations, les métriques de performance des modèles et les analyses de segments pour produire des reportings internes et surveiller l'évolution du churn dans le temps.

3.2 Scénarios d'usage

Scénario 1 Prédiction individuelle : Un responsable CRM saisit les caractéristiques d'un client dans le formulaire de prédiction du dashboard. Il obtient immédiatement la probabilité de churn (ex : 72%), le niveau de risque (élevé), le revenu à risque estimé (ex : 540€) et une recommandation d'action ("Contacter immédiatement et proposer une offre de fidélisation").

Scénario 2 Vue d'ensemble : Un directeur consulte la page d'overview pour visualiser le taux de churn global (10,2%), le nombre de clients à risque (1 021) et le revenu total à risque (862 640€) sur l'ensemble du portefeuille.

Scénario 3 Analyse des données : Un data analyst explore les distributions, les corrélations et les taux de churn par segment pour identifier les profils clients les plus à risque.

3.3 Décisions à soutenir

La solution ne produit pas des prédictions pour le plaisir de la statistique chaque résultat doit déboucher sur une décision concrète. La première décision est de savoir qui contacter en priorité : le score de churn permet de trier les clients par niveau de risque et de concentrer les efforts sur ceux qui en ont le plus besoin. La deuxième décision est de calibrer l'effort de rétention : un client à risque représentant 2 000 € de revenus justifie un investissement commercial plus important qu'un client à 150 €. La troisième décision est d'adapter le discours commercial : grâce à l'interprétabilité SHAP, le responsable CRM sait si le risque vient d'une insatisfaction, d'impayés ou d'une inactivité, et peut adapter son approche en conséquence.

3.4 Hypothèses et contraintes

Plusieurs hypothèses ont guidé la conception de la solution. Nous supposons que les données historiques du dataset sont suffisamment représentatives du comportement réel des clients pour que les patterns appris soient généralisables. Nous supposons également qu'un client présentant un score de churn élevé peut être retenu si une action est menée suffisamment tôt, ce qui justifie l'approche préventive. Enfin, le revenu à risque est approximé par le total_revenue du client cherner dans un contexte réel, il serait calculé à partir des données CRM exactes.

Du côté des contraintes, la solution doit être utilisable par des profils non techniques, ce qui impose une interface graphique claire sans exposition du code. Les prédictions doivent

être explicables car un score opaque sans justification ne sera pas adopté par les équipes CRM. Enfin, le temps de réponse du dashboard doit être suffisamment rapide pour une utilisation en réunion ou lors d'un appel client.

3.5 Indicateurs attendus dans le dashboard

Le dashboard a été conçu pour répondre aux besoins de chaque profil utilisateur, en centralisant en un seul endroit toutes les informations nécessaires à la prise de décision.

La page d'overview affiche les KPIs globaux essentiels : le taux de churn actuel, le nombre de clients à risque et le revenu total à risque. Ces indicateurs donnent en un coup d'œil une vision de la situation du portefeuille client et permettent au directeur financier ou au responsable CRM de calibrer l'urgence des actions à mener.

Les graphiques d'analyse permettent quant à eux d'aller plus loin : distribution du churn par segment, taux de résiliation par variable catégorielle, et feature importance du Random Forest. Ces visualisations donnent au data analyst et au responsable marketing les éléments nécessaires pour comprendre quels profils clients sont les plus à risque et pourquoi.

Le formulaire de prédiction individuelle est la fonctionnalité la plus opérationnelle du dashboard : l'utilisateur saisit les caractéristiques d'un client et obtient en temps réel sa probabilité de churn, son niveau de risque (faible, modéré ou élevé) et une recommandation d'action métier adaptée. C'est l'outil quotidien du responsable CRM avant un appel client ou une réunion de rétention.

Enfin, la page de comparaison des modèles présente les métriques de performance des quatre algorithmes testés (accuracy, recall, F1-score, ROC-AUC pour la classification, MAE et R^2 pour la régression), permettant de comprendre sur quelle base technique reposent les prédictions et de justifier le choix des modèles retenus.

4. Méthodologie de Travail et gestion de Projet

4.1 Organisation du projet

Le projet a été conduit selon une approche itérative inspirée de la méthodologie Agile, organisée en sprints successifs correspondant aux grandes phases du pipeline Data Science. Ce choix s'est imposé naturellement : en Data Science, il est rare de savoir dès le départ exactement quels modèles seront retenus ou quels problèmes de qualité de données se poseront. L'approche itérative permet d'avancer par étapes, de valider chaque phase avant de passer à la suivante, et d'ajuster les choix techniques en fonction des résultats intermédiaires. Chaque phase a été documentée dans un notebook dédié, garantissant la traçabilité et la reproductibilité de l'ensemble du pipeline.

Ce projet a été réalisé en binôme. La répartition des tâches a permis de travailler en parallèle sur certaines phases, notamment la modélisation, où chaque membre a pris en charge deux algorithmes distincts avant de mettre en commun les résultats pour la comparaison finale.

4.2 Grandes étapes

Le projet s'est déroulé en six phases successives, chacune produisant un livrable concret.

La première phase, l'exploration des données (EDA), a consisté à comprendre la structure du dataset, identifier les variables pertinentes, détecter les anomalies et formuler les premières hypothèses métier. Elle a produit un ensemble de visualisations et d'insights documentés dans EDA.ipynb.

La deuxième phase, le prétraitement, a transformé les données brutes en un format exploitable par les algorithmes : traitement des valeurs manquantes, encodage, standardisation et construction du pipeline sklearn sérialisé dans preprocessing.ipynb.

La troisième phase a consisté à entraîner les quatre modèles de Machine Learning et Deep Learning sur les deux tâches prédictives, chacun documenté dans son propre notebook (model_logistic_regression.ipynb, model_random_forest.ipynb, model_xgboost.ipynb, model_mlp.ipynb).

La quatrième phase, la comparaison finale, a centralisé les métriques de tous les modèles, produit le tableau comparatif et appliqué l'analyse SHAP sur le modèle retenu (comparaison_finale.ipynb).

La cinquième phase a consisté à développer le dashboard Streamlit (app.py), en transformant les modèles sérialisés en interface décisionnelle utilisable par des équipes non techniques.

Enfin, la sixième phase a consisté à exposer les modèles via une API REST FastAPI (main.py), avec trois endpoints documentés et une validation Pydantic des données d'entrée.

Un planning détaillé sous forme de mini-Gantt est disponible en annexe (voir Annexe 1), présentant la chronologie de chaque phase et les livrables associés.

En parallèle de la rédaction du rapport, une présentation de soutenance a été préparée, synthétisant les choix techniques, les résultats et la démonstration live du dashboard.

4.3 Outils utilisés

Le projet s'appuie sur un ensemble d'outils couvrant l'ensemble du cycle de développement. Le versionnement du code a été assuré via Git et GitHub, permettant de tracer chaque évolution du projet et de revenir en arrière en cas d'erreur. L'environnement de développement principal est VS Code, complété par Jupyter Notebook pour les phases d'exploration et de modélisation. La gestion des dépendances repose sur pip et un fichier requirements.txt garantissant la reproductibilité de l'environnement. Le suivi des tâches et l'organisation du projet ont été gérés via un tableau de bord personnel, dont le planning détaillé est disponible en Annexe 1.

4.4 Risques rencontrés et solutions

Le premier risque majeur était le déséquilibre des classes : avec seulement 10,2% de churners, un modèle naïf aurait appris à prédire "le client reste" pour tout le monde et atteint 89,8% d'accuracy sans aucune valeur prédictive. Ce problème a été résolu par deux approches complémentaires : un split stratifié pour préserver le ratio dans les jeux d'entraînement et de test, et un RandomUnderSampling appliqué sur le jeu d'entraînement pour équilibrer les classes à 50/50.

Le deuxième risque concernait les valeurs manquantes dans la colonne `complaint_type`, qui comptait 2 045 NaN. Plutôt que de supprimer ces lignes ou d'imputer une valeur arbitraire, nous avons opté pour un remplacement par 'No_Complaint', ce qui correspond à la réalité métier : un client sans plainte enregistrée n'a simplement pas eu de problème à signaler.

Le troisième risque était lié aux performances décevantes du Deep Learning en classification. Malgré l'ajout de couches Dropout, l'optimisation du seuil de décision et plusieurs ajustements d'hyperparamètres, le MLP n'a pas réussi à surpasser le Random Forest sur cette tâche. Plutôt que de forcer son utilisation, nous avons fait le choix rigoureux de le retenir uniquement là où il excelle réellement, c'est-à-dire en régression, et de justifier ce choix par les métriques.

Le quatrième risque était technique : l'intégration du pipeline de preprocessing dans le dashboard et l'API. Pour éviter tout écart entre la transformation appliquée lors de l'entraînement et celle appliquée lors de l'inférence, le preprocessor sklearn complet a été sérialisé avec joblib et rechargé à l'identique dans `app.py` et `main.py`, garantissant une cohérence totale entre les environnements.

5. Référentiel de Données

5.1 Source et description

Le dataset utilisé est le Customer Churn Business Dataset, disponible sur Kaggle. Il représente une entreprise SaaS fictive disposant de 10 000 clients actifs, décrits chacun par 32 variables couvrant l'ensemble de leur relation avec le service. Les données sont transversales, c'est-à-dire qu'elles constituent une photographie à un instant donné et ne comportent pas de dimension temporelle explicite ce point constitue d'ailleurs l'une des limites identifiées du dataset.

Le fichier est au format CSV, avec 10 000 lignes et 32 colonnes, ce qui représente un volume de taille moyenne, suffisant pour entraîner des modèles de Machine Learning fiables tout en restant manipulable sans infrastructure de calcul spécialisée.

5.2 Variables disponibles

Le dataset couvre 5 grandes catégories de variables :

Catégorie	Variables principales	Type
-----------	-----------------------	------

Profil client	gender, age, country, customer_segment, tenure_months	Mixte
Comportement	monthly_logins, weekly_active_days, avg_session_time, features_used	Numérique
Facturation	monthly_fee, total_revenue, payment_failures, discount_applied	Mixte
Support	support_tickets, avg_resolution_time, complaint_type, csat_score	Mixte
Engagement	email_open_rate, marketing_click_rate, nps_score, referral_count	Numérique

Les variables de profil (ancienneté, segment client, type de contrat) décrivent le contexte relationnel du client avec le service. Un client avec une faible ancienneté ou un contrat mensuel est statistiquement plus susceptible de partir qu'un client fidèle sous contrat annuel. Les variables de comportement (connexions mensuelles, jours actifs, fonctionnalités utilisées) mesurent l'engagement réel avec le produit un client peu actif est souvent un client sur le départ. Les variables de facturation (impayés, augmentation de prix récente) capturent les signaux de friction financière qui précèdent fréquemment une résiliation. Les variables de support (tickets ouverts, temps de résolution, score de satisfaction CSAT) reflètent la qualité de l'expérience client et son niveau de frustration. Enfin, les variables d'engagement (NPS, taux d'ouverture des emails, nombre de recommandations) mesurent l'attachement émotionnel au service et la propension à le recommander.

5.3 Variables cibles

Deux variables cibles ont été définies pour les deux tâches prédictives du projet :

- Tâche 1 Classification : la variable churn (0 = le client reste, 1 = le client part) est déjà présente dans le dataset. Elle est particulièrement pertinente d'un point de vue métier car elle représente directement la décision de résiliation que l'entreprise cherche à anticiper.
- Tâche 2 Régression : la variable revenue_at_risk n'existe pas directement dans le dataset. Elle a été construite comme variable dérivée selon la formule $\text{revenue_at_risk} = \text{total_revenue} \times \text{churn}$, ce qui revient à considérer que le revenu total d'un client cherner est entièrement à risque. Cette approximation, bien qu'imparfaite, permet de hiérarchiser les interventions de rétention par impact financier réel plutôt que par simple probabilité de départ.

5.4 Types de données

Le dataset présente trois types de variables :

- 14 colonnes entières (int64) : variables numériques discrètes comme l'âge, le nombre de connexions ou les tickets support
- 6 colonnes décimales (float64) : variables numériques continues comme le taux d'ouverture des emails, le score CSAT ou le taux de croissance d'usage
- 12 colonnes textuelles/catégorielles (object) : variables comme le genre, le pays, le segment client ou le type de plainte, qui nécessiteront un encodage avant modélisation

5.5 Limites du dataset

Plusieurs limites ont été identifiées et prises en compte dans la conception du pipeline :

Le déséquilibre des classes est la limite la plus structurante : 89,8% des clients restent contre seulement 10,2% qui partent. Un modèle naïf qui prédirait systématiquement "le client reste" obtiendrait 89,8% d'accuracy sans aucune valeur prédictive réelle. Cette limite a nécessité des stratégies spécifiques (stratified split, class_weight='balanced', ajustement du seuil de décision).

L'absence de dimension temporelle empêche de modéliser l'évolution du comportement client dans le temps. Dans un contexte réel, il serait précieux de disposer de séries temporelles pour détecter les signaux de dégradation progressive de l'engagement.

La variable `revenue_at_risk` est une approximation construite par proxy plutôt que mesurée directement. Dans un contexte de production, elle devrait être calculée à partir des données CRM réelles, intégrant les contrats, les remises et les engagements contractuels.

Enfin, le dataset peut présenter un biais de sélection : les clients déjà partis avant la constitution du dataset ne sont pas représentés dynamiquement, ce qui peut sous-estimer certains profils de churners.

6. Analyse Exploratoire des Données

6.1 Statistiques descriptives globales

Avant de lancer la moindre modélisation, une analyse exploratoire approfondie a été conduite sur l'ensemble du dataset. Cette étape est fondamentale : elle permet de comprendre la structure des données, d'identifier les anomalies, et de formuler les premières hypothèses métier qui guideront les choix de modélisation.

Le dataset se révèle globalement propre : aucun doublon n'a été détecté sur les 10 000 lignes, et une seule colonne présente des valeurs manquantes `complaint_type` avec 2 045 NaN, soit 20,45% de valeurs absentes. Cette absence n'est pas une erreur de saisie mais une information métier valide : un client sans plainte enregistrée n'a tout simplement pas eu de problème à signaler.

Les statistiques descriptives des principales variables numériques révèlent les caractéristiques suivantes du portefeuille client :

- L'âge des clients est uniformément distribué entre 18 et 74 ans, sans concentration sur une tranche d'âge particulière
- La tenure (ancienneté) varie de 1 à 59 mois de façon uniforme, indiquant un portefeuille client diversifié en termes de maturité relationnelle
- Le `monthly_fee` varie de 10 à 100€, avec une moyenne autour de 55€
- Le `csat_score` (satisfaction) va de 1 à 5, avec une médiane proche de 3, suggérant une satisfaction globale neutre
- Le `nps_score` varie de -100 à +100, reflétant une grande diversité dans le niveau de recommandation

Les visualisations de ces distributions (histogrammes et boxplots) sont disponibles en Annexe 2.

6.2 Distribution de la variable cible — Churn

L'analyse de la variable cible churn révèle un déséquilibre marqué : 8 979 clients restent (89,8%) contre seulement 1 021 clients qui partent (10,2%). Ce déséquilibre est illustré dans les graphiques de l'Annexe 2.

Un modèle naïf qui prédirait systématiquement "le client reste" obtiendrait 89,8% d'accuracy sans aucune valeur prédictive réelle. Cette observation justifie trois décisions techniques importantes : l'utilisation de métriques adaptées (Recall, F1-Score, ROC-AUC plutôt que l'accuracy), l'application du paramètre `class_weight='balanced'` dans les algorithmes, et l'ajustement du seuil de décision au-delà de 0,5.

D'un point de vue financier, les clients qui partent ont un revenu moyen de 845€, contre 1 081€ pour ceux qui restent. Le revenu total à risque s'élève à 862 640€, ce qui représente l'enjeu financier direct que notre système de rétention cherche à protéger.

6.3 Distribution de la variable cible Revenue at Risk

La variable `revenue_at_risk` présente une distribution bimodale très particulière, directement liée au déséquilibre du churn : 89,8% des valeurs sont exactement à 0 (clients qui restent et dont le revenu n'est pas à risque), tandis que les 10,2% restants correspondent aux revenus des churners réels, avec une forte dispersion allant de quelques dizaines d'euros à plusieurs milliers selon le `monthly_fee` du client.

Cette distribution rend la tâche de régression intrinsèquement difficile, ce qui explique les faibles R^2 observés pour tous les modèles testés. La métrique la plus pertinente pour évaluer cette tâche est donc la MAE (erreur absolue moyenne en euros), directement interprétable dans un contexte métier, plutôt que le R^2 qui pénalise fortement les distributions aussi déséquilibrées.

6.4 Insights majeurs par segment

L'analyse du taux de churn par variable catégorielle, représentée sous forme de graphiques en barres (voir Annexe 2), révèle des résultats contrastés selon les variables.

Le segment client est de loin la variable catégorielle la plus discriminante : les clients SME (petites et moyennes entreprises) churnent à 30,9%, soit trois fois plus que les particuliers (Individual : 10,9%) et bien au-dessus de la moyenne globale de 10,2%. Les clients Enterprise présentent quant à eux le taux le plus faible à 9,4%. Ce résultat est le signal le plus fort identifié dans l'EDA et suggère que les PME ont des besoins spécifiques moins bien couverts par l'offre actuelle.

Le genre, le pays et le canal d'acquisition présentent des taux de churn très proches de la moyenne globale (entre 8,4% et 11,5%), sans signal discriminant clair. Ces variables auront donc un faible pouvoir prédictif dans les modèles.

Le type de contrat montre des différences très faibles : Monthly (10,3%), Quarterly (9,9%) et Yearly (10,3%). Contrairement à l'intuition métier initiale, le type de contrat ne semble pas être un facteur déterminant dans ce dataset.

Le type de plainte présente également des écarts minimes : Billing (10,2%), Service (10,0%) et Technical (10,3%). Ces différences sont trop faibles pour constituer un signal prédictif fiable à elles seules.

La réponse aux enquêtes de satisfaction montre une légère tendance : les clients Unsatisfied churnent à 11,1%, contre 10,1% pour les Satisfied et 9,7% pour les Neutral. L'écart est présent mais modéré.

En conclusion, le segment client SME est le seul signal catégoriel vraiment fort identifié dans cette EDA. Les autres variables catégorielles contribueront de façon marginale à la prédiction, ce qui renforce l'importance des variables numériques comme le `csat_score`, le `nps_score` et les `payment_failures` identifiées dans la section suivante.

6.5 Corrélations et relations avec la variable cible

L'analyse de corrélation entre les variables numériques et la variable cible churn révèle des relations faibles mais statistiquement significatives. La heatmap de corrélation complète est disponible en Annexe 2.

Les principales relations identifiées sont les suivantes :

- Le `csat_score` présente une corrélation négative avec le churn : plus la satisfaction est faible, plus le risque de départ est élevé. C'est la variable la plus corrélée à la cible, ce que confirmera l'analyse SHAP dans la section dédiée.
- Le `nps_score` suit la même logique : les détracteurs (score négatif) sont significativement plus susceptibles de churner que les promoteurs (score élevé).

- Les `payment_failures` sont positivement corrélés au churn : chaque échec de paiement supplémentaire augmente la probabilité de résiliation.
- Le `weekly_active_days` est négativement corrélé au churn : un client qui utilise le service régulièrement est un client qui y trouve de la valeur et qui est moins enclin à partir.
- Le `last_login_days_ago` est positivement corrélé : plus le dernier login est ancien, plus le client est potentiellement sur le départ.

Ces corrélations restent modérées (inférieures à 0,3 en valeur absolue), ce qui justifie le recours à des algorithmes capables de capturer des relations non linéaires comme le Random Forest ou le MLP.

6.6 Valeurs extrêmes

L'analyse des valeurs extrêmes révèle que le dataset ne contient pas d'outliers aberrants au sens statistique. Les plages de valeurs sont cohérentes avec la réalité métier d'une entreprise SaaS : le `monthly_fee` va de 10 à 100€, l'`avg_resolution_time` de 1 à 62 heures, et le `nps_score` de -100 à +100. Les variables comportementales comme `monthly_logins` (0 à 54) ou `weekly_active_days` (0 à 7) restent dans des plages plausibles. Aucune suppression d'outliers n'a donc été jugée nécessaire lors du prétraitement.

6.7 Premières hypothèses métier

L'EDA permet de formuler plusieurs hypothèses qui ont guidé la modélisation :

Un client SME sous contrat mensuel, présentant un CSAT faible, des impayés et une faible activité hebdomadaire constitue le profil type le plus à risque de churn. La satisfaction client, mesurée par le `csat_score` et le `nps_score`, apparaît comme le signal le plus discriminant pour anticiper le départ. L'inactivité comportementale, traduite par un faible `weekly_active_days` et un `last_login_days_ago` élevé, précède fréquemment la résiliation.

Enfin, le revenu à risque est concentré sur un nombre limité de clients à fort `monthly_fee`, ce qui renforce l'intérêt d'une approche combinant classification et régression pour prioriser les interventions.

6.8 Note sur les visualisations

Les visualisations produites dans cette phase d'EDA (histogrammes des distributions numériques, graphiques de taux de churn par variable catégorielle, heatmap de corrélation et analyse de la distribution de `revenue_at_risk`) constituent des outils d'analyse scientifique destinés à comprendre les données en profondeur. Elles diffèrent volontairement des visualisations du dashboard final, qui sont orientées utilisateur métier : KPIs synthétiques, graphiques interactifs filtrables et résultats de prédiction en temps réel. Cette distinction entre analyse technique et outil décisionnel est maintenue tout au long du projet, conformément aux consignes du sujet.

7. Préparation et Transformation des Données

7.1 Nettoyage

Le nettoyage a constitué la première étape du prétraitement, guidée par les observations de l'EDA. Deux colonnes ont été supprimées : `customer_id`, qui est un identifiant unique sans aucune valeur prédictive, et `city`, trop granulaire et redondante avec la variable `country`. Après suppression, le dataset passe de 32 à 30 colonnes.

La seule valeur manquante identifiée concernait la colonne `complaint_type`, avec 2 045 NaN soit 20,45% de valeurs absentes. Plutôt que de supprimer ces lignes ou d'imputer une valeur arbitraire, nous avons opté pour un remplacement par 'No_Complaint'. Ce choix est justifié par la logique métier : un client sans plainte enregistrée n'a simplement pas eu de problème à signaler. L'absence de valeur est elle-même une information. Après traitement, le dataset ne contient plus aucune valeur manquante ni doublon.

Concernant les outliers, l'EDA a montré que les valeurs extrêmes identifiées (`total_revenue` jusqu'à 5 900€, `last_login_days_ago` jusqu'à 80 jours, `payment_failures` jusqu'à 5) sont des signaux métier réels et non des erreurs de saisie. La décision a donc été prise de les conserver intégralement dans le pipeline.

7.2 Création de la variable cible de régression

La variable `revenue_at_risk` n'existe pas directement dans le dataset et a été construite par approximation métier. Si un client a churné (`churn == 1`), son revenu total est considéré entièrement à risque : `revenue_at_risk = total_revenue`. Si le client reste (`churn == 0`), le revenu à risque est nul : `revenue_at_risk = 0`.

Cette construction résulte en une distribution fortement asymétrique : 89,8% des valeurs sont à 0 et 10,2% correspondent aux revenus réels des churners. Cette asymétrie explique les faibles R^2 obtenus en régression et justifie le choix de la MAE comme métrique principale pour cette tâche.

7.3 Encodage et standardisation

Le pipeline de preprocessing a été construit avec `ColumnTransformer` de `scikit-learn`, garantissant que les mêmes transformations sont appliquées de façon identique sur le jeu d'entraînement et le jeu de test, sans risque de data leakage.

Les 19 variables numériques ont été standardisées avec `StandardScaler`, qui centre et réduit chaque variable ($\text{moyenne} = 0$, $\text{écart-type} = 1$). Cette standardisation est indispensable pour la Régression Logistique et le MLP, dont les algorithmes d'optimisation sont sensibles aux différences d'échelle entre variables. La visualisation des distributions après standardisation est disponible en Annexe 3 (Figure 1), confirmant que chaque variable est bien centrée autour de 0. Elle est en revanche sans effet sur Random Forest et XGBoost, qui reposent sur des comparaisons par seuils et sont naturellement invariants à l'échelle.

Les 10 variables catégorielles ont été encodées avec `OneHotEncoder`. Ce choix est justifié par la faible cardinalité de ces variables (entre 2 et 7 modalités par colonne), qui ne risque pas de générer un nombre excessif de colonnes. Après encodage, le dataset passe de 29 features à 51 features, ce qui constitue l'espace d'entrée final de tous les modèles.

Le pipeline complet (StandardScaler + OneHotEncoder) a été sérialisé avec joblib sous le nom `preprocessor.pkl`, permettant de le réutiliser à l'identique dans le dashboard Streamlit et l'API REST FastAPI.

7.4 Split Train/Test

Le dataset a été divisé selon un split stratifié 80/20 sur la variable `churn`, garantissant que le ratio de churners est préservé dans les deux jeux :

- Jeu d'entraînement : 8 000 clients 10,2% de churners
- Jeu de test : 2 000 clients 10,2% de churners

Le split stratifié est essentiel dans notre contexte de classes déséquilibrées : un split aléatoire classique pourrait par malchance concentrer trop de churners dans l'un des jeux et biaiser l'évaluation. Le StandardScaler a été fitté uniquement sur le jeu d'entraînement et appliqué (transform) sur le jeu de test, évitant tout data leakage.

Un contrôle anti-data leakage a été effectué en fin de notebook, vérifiant que les jeux train et test sont bien disjoints, que le StandardScaler a bien été fitté sur le train uniquement, et que la stratification est correcte. Tous les contrôles sont passés avec succès.

Aucune validation croisée n'a été utilisée dans ce projet. Le volume de données (10 000 clients) est suffisant pour que le jeu de test de 2 000 clients fournisse une estimation fiable des performances.

7.5 Gestion du déséquilibre des classes

Le déséquilibre 89,8% / 10,2% a été traité par un RandomUnderSampling appliqué sur le jeu d'entraînement uniquement. Cette technique réduit la classe majoritaire (non-churners) pour obtenir un équilibre parfait 50/50, faisant passer le jeu d'entraînement de 8 000 à 1 634 samples. Le jeu de test reste inchangé dans sa distribution réelle, garantissant une évaluation réaliste des performances. La distribution avant et après undersampling est illustrée en Annexe 3 (Figure 2).

8. pipeline IA et Architecture

8.1 Vue d'ensemble du pipeline

Le pipeline Data Science de bout en bout couvre l'intégralité de la chaîne de traitement, depuis les données brutes jusqu'aux outils déployables. Le schéma complet est présenté en Annexe 4.

Le point de départ est le fichier CSV issu de Kaggle (10 000 lignes × 32 colonnes), chargé avec Pandas. La première étape est l'analyse exploratoire des données (EDA), réalisée avec Pandas, Matplotlib et Seaborn, qui produit les insights et visualisations documentés dans `EDA.ipynb`. Le nettoyage qui suit transforme le dataset de 32 à 30 colonnes (suppression de `customer_id` et `city`, traitement des valeurs manquantes de `complaint_type`).

Le preprocessing construit avec ColumnTransformer de scikit-learn applique le StandardScaler sur les 19 variables numériques et le OneHotEncoder sur les 10 variables catégorielles, produisant un espace de 51 features. Le pipeline est sérialisé dans preprocessor.pkl. Le dataset est ensuite divisé par un split stratifié 80/20, donnant 8 000 clients pour l'entraînement et 2 000 pour le test, avec préservation du ratio de churners.

La phase de modélisation entraîne 4 algorithmes sur 2 tâches en parallèle : Régression Logistique, Random Forest, XGBoost et MLP pour la classification binaire (churn), et les mêmes 4 algorithmes pour la régression (revenue_at_risk). L'évaluation comparative centralise les métriques de tous les modèles via sklearn.metrics, produisant le tableau comparatif final. L'interprétabilité est assurée par SHAP, appliqué sur le Random Forest pour produire la feature importance globale et les explications locales par client.

Les modèles retenus (Random Forest pour la classification, MLP pour la régression) sont sérialisés avec joblib (.pkl) et Keras (.keras). Ces fichiers sont ensuite chargés par deux outils distincts : le dashboard Streamlit (app.py) qui expose une interface décisionnelle en 4 pages, et l'API REST FastAPI (main.py) qui expose 3 endpoints documentés (/health, /predict, /model-info).

Étape	Outil / Technologie	Sortie
Données brutes	CSV — Kaggle	10 000 × 32
EDA	Pandas, Matplotlib, Seaborn	Insights + visualisations
Nettoyage	Pandas	Dataset propre (10 000 × 30)
Preprocessing	Scikit-learn Pipeline	preprocessor.pkl (51 features)
Split	train_test_split stratifié	8 000 train / 2 000 test
Modèles ML	Sklearn, XGBoost, Keras	4 modèles × 2 tâches
Évaluation	Sklearn metrics	Tableau comparatif
Interprétabilité	SHAP	Feature importance globale + locale
Sérialisation	joblib + .keras	Modèles sauvegardés
Dashboard	Streamlit (app.py)	4 pages interactives
API REST	FastAPI (main.py)	3 endpoints

8.2 Architecture technique

L'architecture de la solution repose sur trois composants principaux qui interagissent autour des modèles sérialisés.

Le backend ML est constitué des modèles sérialisés dans le dossier /models/ : rf_classification.pkl (Random Forest pour la classification), mlp_regression.keras (MLP pour la régression) et preprocessor.pkl (pipeline de transformation). Ces fichiers constituent le cœur de la solution et sont partagés entre le dashboard et l'API.

Le dashboard Streamlit (app.py) charge les modèles au démarrage via le décorateur `@st.cache_resource`, qui évite de les recharger à chaque interaction utilisateur. Il expose une interface graphique en 4 pages accessibles depuis une sidebar de navigation.

L'API REST FastAPI (main.py) est servie par Uvicorn comme serveur ASGI. Elle charge les mêmes modèles au démarrage et les expose via 3 endpoints documentés automatiquement par la documentation Swagger intégrée à FastAPI.

8.3 Organisation du dépôt

- /api/ — main.py (FastAPI)
- /dashboard/ — app.py (Streamlit)
- /data/ — customer_churn_business_dataset.csv
- /models/ — tous les modèles sérialisés
- /notebooks/ — EDA, preprocessing, 4 modèles, comparaison
- /results/ — courbes ROC, matrices de confusion, métriques CSV

Voir annexe 4

9. Implémentation Technique

9.1 Langages et librairies utilisés

Le projet repose entièrement sur Python 3.11, choisi pour sa richesse en librairies Data Science et sa compatibilité avec l'ensemble des outils utilisés. Les principales librairies sont les suivantes :

- Pandas et NumPy : manipulation et transformation des données
- Scikit-learn : pipeline de preprocessing, modèles de Machine Learning, métriques d'évaluation
- TensorFlow / Keras : construction et entraînement du MLP (Deep Learning)
- XGBoost : gradient boosting optimisé
- imbalanced-learn : RandomUnderSampler pour la gestion du déséquilibre
- SHAP : interprétabilité des prédictions du Random Forest
- Matplotlib, Seaborn, Plotly : visualisations EDA et dashboard
- Streamlit : dashboard interactif
- FastAPI + Uvicorn : API REST
- joblib : sérialisation des modèles

L'organisation du dépôt Git et les fichiers principaux sont décrits en section 8.3.

9.2 Modèles testés et justification

Pour chaque tâche, 4 modèles ont été entraînés sur le même jeu d'entraînement (1 634 samples après undersampling pour la classification, 8 000 pour la régression) et évalués sur le même jeu de test (2 000 clients). Ce protocole d'évaluation strict garantit la comparabilité des résultats.

Tâche 1 Classification (prédire le churn)

La Régression Logistique a été retenue comme modèle baseline. Sa simplicité et son interprétabilité en font un point de référence indispensable : si un modèle complexe ne fait pas mieux qu'une régression logistique, sa complexité n'est pas justifiée.

Le Random Forest est un ensemble de 200 arbres de décision entraînés en parallèle. Sa robustesse aux outliers, sa capacité à capturer des relations non linéaires et sa feature importance native en font un modèle particulièrement adapté à notre contexte. Il ne nécessite pas de standardisation des données et gère naturellement les variables mixtes.

XGBoost est un algorithme de boosting par gradient qui construit les arbres séquentiellement, chacun corrigeant les erreurs du précédent. Il est généralement très performant sur des données tabulaires mais moins interprétable que le Random Forest.

Le MLP (Deep Learning) est un réseau de neurones multicouches capable de capturer des relations très complexes et non linéaires. Son inclusion répond à l'exigence du projet d'intégrer au moins un modèle de Deep Learning.

Tâche 2 — Régression (prédire le revenu à risque)

La Ridge Regression constitue le baseline régularisé pour la tâche de régression. Le Random Forest Regressor et XGBoost Regressor sont les équivalents en régression des modèles testés en classification. Le MLP Regressor est le modèle Deep Learning retenu pour cette tâche.

9.3 Hyperparamètres principaux — Random Forest Classification

Paramètre	Valeur	Justification
n_estimators	200	Nombre d'arbres — bon compromis performance/temps
max_depth	10	Limite la profondeur pour éviter l'overfitting
class_weight	balanced	Compense le déséquilibre 90/10%
min_samples_leaf	5	Régularisation pour la généralisation

9.4 Architecture MLP — Régression

Le MLP retenu pour la régression est construit avec Keras et suit l'architecture suivante :

- Input : 51 features (après preprocessing)
- Dense(128) + ReLU + Dropout(0.3)
- Dense(64) + ReLU + Dropout(0.2)
- Dense(32) + ReLU
- Dense(1) + activation linéaire (sortie continue)
- Optimizer : Adam (lr = 0.001) — Loss : MSE — Callbacks : EarlyStopping + ReduceLROnPlateau

Les couches Dropout ont été ajoutées spécifiquement pour limiter le risque d'overfitting, qui est élevé sur un réseau de cette profondeur appliqué à un dataset de taille modeste. L'EarlyStopping arrête l'entraînement dès que la loss de validation ne s'améliore plus, évitant ainsi de surapprendre sur le jeu d'entraînement.

9.5 Analyse critique du Deep Learning

Le guide insiste sur le fait que le Deep Learning ne doit pas être intégré de manière automatique. Dans notre projet, cette analyse critique a effectivement guidé nos choix.

Pour la **classification**, le MLP s'est révélé inférieur au Random Forest sur toutes les métriques clés (ROC-AUC, Recall, F1-Score). Plusieurs facteurs expliquent ce résultat : le dataset de 10 000 clients est de taille modeste pour un réseau de neurones, la variable cible est fortement déséquilibrée, et les relations entre variables sont suffisamment capturées par un modèle ensembliste sans nécessiter une architecture profonde. Le coût computationnel du MLP (temps d'entraînement significativement plus long) n'est donc pas justifié pour cette tâche.

Pour la **régression**, le MLP obtient la meilleure MAE (136,89€) et s'impose comme le modèle retenu. Sa capacité à capturer des relations non linéaires entre les 51 features et le revenu à risque lui donne un avantage sur les modèles plus simples. Cependant, le R^2 reste très faible pour tous les modèles (autour de 0,07), ce qui confirme que la distribution très asymétrique de `revenue_at_risk` (89,8% de valeurs à 0) constitue une limite intrinsèque difficile à surmonter quelle que soit l'architecture utilisée.

10. évaluation Comparative des modèles

10.1 Classification — Comparaison des 4 modèles

Tous les modèles ont été évalués sur le même jeu de test (2 000 clients, dont 204 churners réels soit 10,2%). Les métriques prioritaires dans notre contexte métier sont le Recall (éviter les faux négatifs ne pas manquer un churner) et le ROC-AUC (performance discriminante globale). L'accuracy seule est trompeuse dans ce contexte déséquilibré : un modèle prédisant systématiquement "le client reste" obtiendrait 89,8% d'accuracy sans aucune valeur prédictive.

Modèle	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	0.714	0.243	0.853	0.379	0.810
XGBoost	0.713	0.230	0.775	0.355	0.781
Régression Logistique	0.655	0.174	0.637	0.274	0.709
MLP (Deep Learning)	0.645	0.135	0.461	0.209	0.560

10.2 Régression — Comparaison des 4 modèles

Modèle	MAE (€)	RMSE (€)	R ²
MLP (Deep Learning)	136.89	372.30	0.068
Random Forest	145.39	373.57	0.061
Ridge Regression	156.11	375.15	0.053
XGBoost	158.61	386.03	-0.002

10.3 Analyse et justification des choix

Random Forest pour la classification

Le Random Forest est retenu comme meilleur modèle de classification sur la base de plusieurs critères complémentaires.

En termes de performance, il obtient le meilleur ROC-AUC (0,810), le meilleur Recall (0,853) et le meilleur F1-Score (0,379). Identifier 85,3% des churners réels est crucial dans notre contexte métier : chaque cherner manqué représente en moyenne 845€ de revenu perdu.

En termes de stabilité et généralisation, le Random Forest est par nature un modèle ensembliste qui moyenne les prédictions de 200 arbres indépendants, ce qui le rend beaucoup moins sensible au bruit que les modèles individuels. Son risque d'overfitting est limité par les paramètres `max_depth=10` et `min_samples_leaf=5`.

En termes d'interprétabilité, il fournit une feature importance native directement exploitable par les équipes CRM pour comprendre quels signaux précèdent le départ d'un client.

En termes de facilité de déploiement, il se sérialise simplement avec `joblib` en un fichier `.pkl` léger, chargeable en une ligne de code dans le dashboard et l'API.

Pourquoi pas XGBoost ? Son Recall est significativement plus faible (0,775 vs 0,853), ce qui signifie qu'il manquerait environ 16 churners supplémentaires sur 100 détectés par le Random Forest. Dans un contexte où chaque client manqué représente 845€ perdus, ce gap est inacceptable.

Pourquoi pas le MLP pour la classification ? Son ROC-AUC de 0,560 trahit une faible capacité discriminante globale. Malgré l'ajustement du seuil de décision, le MLP n'apporte aucune valeur ajoutée par rapport aux méthodes ensemblistes sur cette tâche, pour un coût computationnel bien supérieur.

MLP pour la régression

Le MLP est retenu pour la régression car il obtient la meilleure MAE (136,89€), soit une erreur moyenne de prédiction de 136,89€ par client. Sa capacité à capturer des relations non linéaires entre les 51 features et le revenu à risque lui confère un avantage sur les modèles plus simples.

Le R^2 faible (0,068) pour tous les modèles s'explique par la nature très déséquilibrée de la variable cible (89,8% de valeurs exactement à 0). Cette limite est intrinsèque au dataset et non au modèle. Dans ce contexte, la MAE reste la métrique la plus pertinente et la plus directement interprétable pour les équipes métier.

XGBoost obtient un R^2 négatif (-0,002), ce qui signifie qu'il performe moins bien qu'un modèle naïf prédisant la moyenne signe d'une instabilité sur cette distribution très asymétrique.

11. Interprétabilité et Analyse Métier des Résultats

11.1 Feature Importance — Random Forest

L'analyse de la feature importance du Random Forest permet d'identifier les variables qui contribuent le plus aux prédictions du modèle. Ces importances sont calculées en mesurant la réduction moyenne de l'impureté apportée par chaque variable à travers les 200 arbres de la forêt.

Rang	Variable	Importance	Interprétation métier
1	csat_score	0.162	La satisfaction client est le signal le plus fort
2	nps_score	0.145	Les détracteurs chument massivement
3	tenure_months	0.098	Les clients récents partent plus facilement
4	payment_failures	0.087	Les impayés précèdent souvent la résiliation
5	monthly_fee	0.076	Sensibilité prix des clients à risque
6	weekly_active_days	0.071	L'engagement usage prédit la rétention
7	avg_resolution_time	0.065	Résolution lente des tickets → frustration
8	last_login_days_ago	0.058	L'inactivité est un signal de départ imminent

11.2 Lecture métier des résultats — en langage non spécialistealyse SHAP

Pour un responsable CRM ou un directeur marketing qui n'est pas data scientist, voici ce que ces résultats signifient concrètement.

Le facteur le plus déterminant pour prédire le départ d'un client est son niveau de satisfaction. Un client qui note le service 1 ou 2 sur 5 est beaucoup plus susceptible de partir qu'un client satisfait. C'est un signal d'alarme qui doit déclencher une action immédiate de la part des équipes CRM, avant même que le client ne prenne sa décision de résilier.

Le deuxième signal le plus fort est le NPS (Net Promoter Score). Un client qui répond négativement à la question "recommanderiez-vous ce service ?" est un client qui a déjà mentalement décroché. Ces clients détracteurs doivent être identifiés rapidement et faire l'objet d'un suivi personnalisé.

L'ancienneté du client est également très révélatrice. Les clients qui ont souscrit il y a moins de 6 mois sont les plus fragiles : ils n'ont pas encore développé une habitude d'usage forte et peuvent facilement partir si leur expérience initiale est décevante. Les premiers mois sont donc critiques pour la rétention.

Les impayés sont un signal financier direct : un client qui rate plusieurs paiements consécutifs est souvent un client qui cherche à partir ou qui traverse des difficultés. Chaque échec de paiement doit déclencher une prise de contact proactive.

Enfin, l'inactivité comportementale est un précurseur silencieux du churn : un client qui se connecte moins de 2 fois par semaine ou dont le dernier login remonte à plusieurs semaines est un client qui n'utilise plus le service et risque de le résilier à la prochaine occasion.

11.3 Analyse SHAP

Les valeurs SHAP (SHapley Additive exPlanations) ont été calculées sur un échantillon de 500 clients du jeu de test. Contrairement à la feature importance globale qui donne une vision agrégée, SHAP permet d'expliquer chaque prédiction individuellement en quantifiant la contribution de chaque variable pour ce client précis.

Les insights globaux SHAP confirment et précisent les résultats de la feature importance :

Un `csat_score` inférieur à 2 augmente la probabilité de churn de 15 à 25 points de pourcentage par rapport à un client satisfait. Un `nps_score` très négatif (inférieur à -50) est systématiquement associé à une probabilité de churn élevée. Un `tenure_months` inférieur à 6 mois augmente fortement le risque, confirmant la fragilité des nouveaux clients. Plusieurs `payment_failures` consécutifs constituent un signal d'alarme critique qui cumule les effets négatifs. Enfin, un `weekly_active_days` inférieur à 2 jours par semaine est significativement associé à un risque de départ plus élevé.

11.4 Exemple de prédiction individuelle SHAP

Pour illustrer concrètement l'apport de SHAP, voici l'analyse d'un client réel du jeu de test (index 17) dont le churn réel est confirmé :

- Probabilité de churn prédite par le modèle : 55,8% prédiction correcte
- Variables qui aggravent le risque : `csat_score` bas, `payment_failures` = 2, `tenure_months` très court

- Variables qui protègent partiellement : features_used élevé (le client utilise beaucoup de fonctionnalités), nps_score neutre

En termes métier, ce client est un nouveau client (ancienneté courte) qui utilise activement le service mais qui est insatisfait et a déjà eu des problèmes de paiement. Le profil type d'un client à contacter en urgence avec une offre commerciale adaptée plutôt qu'un simple appel de satisfaction.

12. Interface Utilisateur Dashboard Décisionnel

12.1 Objectif et conception

Le dashboard Streamlit a été conçu comme un outil décisionnel destiné à l'utilisateur métier, non comme une simple reproduction des graphiques EDA. Il est organisé en 4 pages accessibles depuis une sidebar de navigation.

12.2 Pages du dashboard

Page 1 — Vue d'ensemble

Affiche les KPIs globaux en temps réel :

- Total Clients : 10 000
- Clients à Risque : 1 021 (10,2% du total)
- Revenu à Risque : 862 640 €
- Revenu moyen global

Complété par des graphiques de distribution du churn et une vue des segments les plus à risque.

Page 2 Analyse des données

Permet d'explorer interactivement les distributions des variables, les taux de churn par segment, les corrélations et les comportements moyens des clients churners vs non-churners. Chaque graphique est accompagné d'une lecture métier.

Page 3 Comparaison des modèles

Présente le tableau comparatif des 4 algorithmes testés (classification et régression), les courbes ROC, les matrices de confusion et la feature importance du Random Forest.

Page 4 Prédiction client

Interface de prédiction en temps réel : l'utilisateur saisit les caractéristiques d'un client (28 variables dans des sliders et menus déroulants). Le dashboard affiche :

- La probabilité de churn (ex : 72%)
- Le niveau de risque (faible / modéré / élevé) avec un code couleur
- Le revenu à risque estimé par le MLP
- Une recommandation d'action métier adaptée au niveau de risque

12.3 Technologies

- Framework : Streamlit
- Visualisations : Matplotlib, Seaborn, Plotly
- Chargement des modèles : `@st.cache_resource` pour éviter les rechargements
- Inférence en temps réel : même pipeline sklearn que l'entraînement

13. API REST

13.1 Rôle et technologie

L'API REST développée avec FastAPI permet d'exposer les modèles comme un service web consommable par n'importe quelle application externe (CRM, application mobile, autre dashboard). Elle constitue la brique d'industrialisation de la solution.

L'API est indépendante du dashboard Streamlit mais complémentaire : là où le dashboard est destiné à un usage humain via une interface graphique, l'API permet une intégration programmatique dans des systèmes tiers comme un CRM, une application mobile ou un autre outil analytique. Les deux partagent les mêmes modèles sérialisés dans `/models/`, garantissant des prédictions identiques quelle que soit la voie d'accès. Voir annexe 7

13.2 Endpoints disponibles

Méthode	Endpoint	Description
GET	/health	Vérifie que l'API est active et que les modèles sont chargés
POST	/predict	Prédit le churn + revenu à risque pour un client donné
GET	/model-info	Retourne les informations sur les modèles utilisés

13.3 Format des données

Le endpoint `/predict` accepte en entrée un objet JSON décrivant un client (28 variables avec validation Pydantic). Il retourne :

- `proba_churn` : probabilité de résiliation (0 à 1)
- `churn_prediction` : 0 (reste) ou 1 (part)
- `risk_level` : faible / modéré / élevé
- `revenue_at_risk` : revenu estimé à risque en € (MLP)
- `recommendation` : action recommandée selon le niveau de risque

13.4 Gestion des erreurs

- HTTP 503 : si les modèles ne sont pas chargés (fichiers .pkl/.keras absents)
- HTTP 422 : si les données d'entrée sont invalides (Pydantic validation)
- Documentation automatique : disponible via /docs (Swagger UI) et /redoc

13.5 Lancement

Commande de lancement : `uvicorn main:app --reload --port 8000`

14. Résultats et Test de Démonstration

14.1 Démonstration réelle sur le dashboard

Le test de démonstration a été réalisé directement sur le dashboard en production. Les captures d'écran ci-dessous montrent le formulaire de saisie complet et le résultat obtenu.

Le profil client saisi est le suivant : homme de 35 ans, Germany, segment Individual, ancienneté 12 mois, canal Web, contrat Monthly, 15 connexions/mois, 3 jours actifs/semaine, session de 15 minutes, 5 features utilisées, dernière connexion il y a 5 jours, abonnement 30€/mois, revenu total 4 373€, paiement par carte, 0 échec de paiement, aucune remise, aucune hausse de prix récente, 1 ticket support, résolution en 24h, No_Complaint, CSAT = 3,5, 0 escalade, taux ouverture email 0,50, taux clic marketing 0,25, NPS = 20, Satisfied, 1 référence.

Voir annexe 8

Résultat obtenu :

- Probabilité de churn : **30,2%**
- Revenu à risque : **123,22 EUR**
- Niveau de risque : **MODÉRÉ**
- Voir annexe 8

Ce résultat est parfaitement cohérent avec le profil saisi. Le client présente des signaux mixtes : son CSAT de 3,5 et son NPS de 20 sont corrects mais pas excellents, son contrat est mensuel (facteur de risque), son ancienneté est courte (12 mois) mais il n'a aucun impayé et reste actif. Le modèle lui attribue logiquement un score modéré de 30,2%, positionné entre le seuil de faible risque (< 30%) et le seuil de risque élevé (> 50%).

La recommandation associée est : *"Surveillance recommandée — Planifier un appel de satisfaction sous 7 jours"*, ce qui constitue une action concrète et proportionnée au niveau de risque détecté.

14.2 Impact financier mesuré

En appliquant le modèle Random Forest sur le jeu de test complet (2 000 clients, 204 churners réels) :

- Churners correctement identifiés : 174 sur 204 (Recall = 85,3%)
- Faux positifs : environ 500 clients signalés à tort comme à risque (Precision = 24,3%)
- Gain estimé si campagne de rétention réussie à 50% sur les vrais churners détectés : ~73 500€ sur le test set
- Gain estimé sur la totalité du portefeuille (10 000 clients) : ~367 500€

Ces chiffres illustrent concrètement la valeur métier de la solution : même avec une précision modérée, le gain financier potentiel justifie largement le coût d'une campagne de rétention ciblée.

14.3 Discussion du comportement du modèle

Le scénario testé confirme la cohérence des prédictions avec l'intuition métier. Un client avec des signaux mixtes obtient un score modéré, ni trop bas ni trop haut, ce qui est exactement le comportement attendu d'un modèle bien calibré.

Deux cas limites ont été identifiés lors des tests. Un client très insatisfait (CSAT = 1) mais avec une très longue ancienneté (tenure > 48 mois) peut obtenir un score modéré plutôt qu'élevé, car l'ancienneté joue un rôle protecteur significatif. De même, un client SME récent mais très actif peut obtenir un score plus faible qu'attendu, car l'engagement comportemental compense partiellement le risque lié au segment.

La principale limite observée est la précision (Precision = 0,243) : sur 100 clients signalés comme à risque, seulement 24 chuneront réellement. Ce compromis est délibéré — dans notre contexte, manquer un chuner réel est plus coûteux que contacter inutilement un client fidèle.

15. Gouvernance, responsabilité et Limites

15.1 Qualité des données

Le dataset utilisé provient de Kaggle et constitue une simulation d'un portefeuille client SaaS. Bien qu'il soit représentatif des problématiques réelles de rétention, il peut ne pas refléter parfaitement la réalité d'une entreprise spécifique, notamment en termes de distribution des segments, de politique tarifaire ou de comportements d'usage propres à un secteur donné.

La variable `revenue_at_risk` est une approximation construite par proxy : elle suppose que le revenu total d'un client chuner est entièrement perdu, ce qui est une simplification. Dans un contexte réel, cette variable devrait être calculée à partir des données CRM exactes, intégrant les contrats en cours, les engagements pluriannuels et les éventuelles pénalités de résiliation.

L'absence de dimension temporelle est également une limite structurante : le dataset représente une photographie à un instant donné et ne permet pas de modéliser l'évolution du comportement client dans le temps. Des séries temporelles permettraient de détecter des signaux de dégradation progressive bien avant qu'ils n'atteignent un seuil critique.

Sur le plan de la traçabilité, le pipeline est entièrement reproductible : chaque étape est documentée dans un notebook dédié, les modèles sont versionnés et sérialisés, et le preprocessor est sauvegardé avec `joblib` pour garantir la cohérence entre l'entraînement et l'inférence.

15.2 Risques de biais

Plusieurs risques de biais ont été identifiés dans la solution. Le déséquilibre des classes (89,8% / 10,2%) est le plus structurant : sans compensation, les modèles auraient naturellement

tendance à prédire la majorité. Ce risque a été traité par le RandomUnderSampling et les stratégies de régularisation décrites dans la section preprocessing.

Le segment SME présente un taux de churn de 30,9%, soit trois fois la moyenne globale. Le modèle peut donc être plus précis pour ce segment que pour les autres, ce qui pourrait introduire un biais dans les prédictions sur les clients Enterprise ou Individual.

Les variables de satisfaction (CSAT, NPS) sont subjectives et dépendent de la volonté des clients de répondre aux enquêtes. Les clients qui répondent peuvent ne pas être représentatifs de l'ensemble du portefeuille, ce qui peut biaiser les distributions observées.

15.3 Robustesse et risque de surapprentissage

La robustesse du modèle a été assurée par plusieurs mécanismes complémentaires. Le Random Forest utilise `min_samples_leaf=5` et `max_depth=10` pour limiter la complexité des arbres individuels et réduire le risque d'overfitting. Le MLP bénéficie de couches Dropout (0.3 et 0.2) et d'un EarlyStopping qui arrête l'entraînement dès que la performance sur la validation ne s'améliore plus.

La validation sur un jeu de test stratifié et disjoint du jeu d'entraînement garantit que les métriques rapportées reflètent la performance réelle sur des données non vues. Un contrôle anti-data leakage a été effectué en fin de preprocessing pour s'assurer que le scaler a bien été fitté uniquement sur le train set.

15.4 Limites de l'interprétabilité

L'analyse SHAP a été appliquée uniquement sur le Random Forest, qui est le modèle retenu pour la classification. Le MLP retenu pour la régression ne bénéficie pas d'une analyse SHAP équivalente, car les méthodes d'explicabilité pour les réseaux de neurones (SHAP DeepExplainer) sont plus complexes à mettre en œuvre et n'ont pas été intégrées dans ce projet. Cette limite réduit la transparence des prédictions de revenu à risque.

Par ailleurs, la feature importance du Random Forest mesure la contribution globale de chaque variable sur l'ensemble du dataset, mais ne garantit pas que cette contribution soit stable sur tous les sous-segments. Une variable importante globalement peut être non pertinente pour un segment spécifique.

15.5 Déploiement responsable

Si cette solution était déployée en production, plusieurs recommandations s'imposeraient. Un système de monitoring des prédictions permettrait de détecter la dérive des données (data drift) dans le temps : si les distributions des variables en production s'éloignent significativement de celles du dataset d'entraînement, les performances du modèle pourraient se dégrader sans que cela soit visible immédiatement.

Une validation humaine reste indispensable dans les cas critiques : un score de churn élevé ne doit pas déclencher automatiquement une action commerciale sans validation par un responsable CRM. Le modèle est un outil d'aide à la décision, pas un système de décision autonome.

Le modèle devrait être réentraîné tous les 3 à 6 mois avec de nouvelles données pour maintenir sa pertinence. Enfin, les données clients étant sensibles, un chiffrement des données

en transit et au repos, ainsi qu'un contrôle d'accès strict aux modèles et aux prédictions, seraient indispensables dans un contexte de production réel.

16. Limites et Piste d'amélioration

16.1 Ce qui a bien fonctionné

Plusieurs aspects du projet ont donné des résultats satisfaisants. Le Random Forest s'est imposé comme un modèle solide et cohérent pour la classification, avec un Recall de 0,853 et un ROC-AUC de 0,810 qui permettent d'identifier la grande majorité des churners réels. L'analyse SHAP a produit des résultats interprétables et cohérents avec l'intuition métier, confirmant le rôle central de la satisfaction client et de l'ancienneté dans la prédiction du churn. Le pipeline de preprocessing est robuste et reproductible, sérialisé avec joblib et réutilisable à l'identique dans le dashboard et l'API. Le dashboard Streamlit offre une interface claire et opérationnelle, avec une prédiction en temps réel et une visualisation intuitive du niveau de risque.

16.2 Ce qui a été difficile

Plusieurs aspects ont posé des défis significatifs. La gestion du déséquilibre des classes (89,8% / 10,2%) a nécessité de nombreux ajustements : choix du RandomUnderSampling, ajustement du seuil de décision, sélection des métriques adaptées. Sans ces ajustements, les modèles auraient produit des résultats trompeurs avec une accuracy élevée mais aucune valeur prédictive réelle.

La tâche de régression sur `revenue_at_risk` s'est révélée intrinsèquement difficile à cause de la distribution très déséquilibrée de la variable cible (89,8% de valeurs à 0). Tous les modèles obtiennent un R^2 très faible, ce qui signifie qu'aucun algorithme ne parvient à modéliser correctement cette variable dans sa forme actuelle. Ce résultat aurait nécessité une reformulation de la variable cible pour être vraiment exploitable.

L'intégration du Deep Learning a également posé des difficultés : le MLP ne surpasse pas le Random Forest en classification malgré plusieurs tentatives d'optimisation (Dropout, ajustement du seuil, EarlyStopping). Cela illustre une réalité importante en Data Science : un modèle plus complexe n'est pas nécessairement plus performant sur des données tabulaires de taille modeste.

16.3 Limites identifiées

Le R^2 faible pour la régression (0,068) est la limite la plus structurante : la variable `revenue_at_risk`, avec 89,8% de valeurs à 0, est trop déséquilibrée pour être bien prédite par un modèle de régression classique. L'absence de données temporelles empêche de modéliser l'évolution du comportement client dans le temps, ce qui limiterait la détection de signaux de dégradation progressive. L'interprétabilité du MLP est limitée car SHAP n'a pas été appliqué au réseau de neurones utilisé pour la régression. Enfin, la solution tourne en local sans déploiement cloud, ce qui limite sa scalabilité et sa disponibilité dans un contexte de production réel.

16.4 Pistes d'amélioration

Plusieurs améliorations significatives pourraient être apportées à la solution. L'enrichissement des données avec des séries temporelles comportementales (évolution du taux d'usage sur 3 mois, tendance du CSAT) permettrait de capturer des signaux de dégradation progressive bien plus tôt. L'optimisation des hyperparamètres via GridSearchCV ou Optuna pourrait améliorer les performances des modèles, notamment pour le XGBoost qui a montré des résultats proches du Random Forest. L'ajout de SHAP DeepExplainer sur le MLP de régression apporterait une interprétabilité complète à l'ensemble de la solution.

Sur le plan de l'infrastructure, un déploiement cloud via Docker sur AWS, GCP ou Azure rendrait la solution scalable et haute disponibilité. L'intégration d'une base de données PostgreSQL permettrait de stocker les prédictions et de suivre l'évolution des scores dans le temps, ouvrant la voie à un véritable système de monitoring du churn. L'intégration de MLflow permettrait de tracker les expériences et de détecter la dérive des données dès qu'elle se produit. Enfin, des modèles plus avancés comme LightGBM, CatBoost ou des architectures Transformer pour les séries temporelles pourraient être explorés si des données temporelles venaient à être disponibles.

17. Conclusion

Ce projet a permis de développer un système intelligent de rétention client complet, allant des données brutes jusqu'à une solution opérationnelle déployable. Nous avons construit un pipeline Data Science rigoureux couvrant toutes les étapes attendues : analyse exploratoire, préparation des données, modélisation multi-algorithmes (4 modèles $\times$ 2 tâches), évaluation comparative, interprétabilité SHAP, dashboard décisionnel et API REST.

Le modèle retenu pour la classification Random Forest avec $\text{ROC-AUC} = 0,810$ et $\text{Recall} = 0,853$ permet d'identifier 85% des clients susceptibles de partir, représentant un enjeu financier de 862 640€ sur un portefeuille de 10 000 clients. Le MLP, retenu pour la régression, permet d'estimer le revenu à risque avec une erreur moyenne de 136,89€

L'objectif du projet n'était pas uniquement d'obtenir le meilleur score, mais de construire une solution robuste, explicable, défendable et exploitable dans un contexte professionnel réel. Cette démarche s'est traduite par :

- Un pipeline reproductible et documenté dans des notebooks structurés
- Une sérialisation des modèles permettant leur réutilisation dans le dashboard et l'API
- Une interprétabilité SHAP permettant de justifier chaque prédiction auprès des équipes métier
- Un dashboard conçu pour l'utilisateur final, pas pour le data scientist
- Une API REST exposant la solution comme un service industrialisable

Ce projet a permis de développer des compétences couvrant l'ensemble du spectre Data Science : exploration et préparation des données, Machine Learning supervisé, Deep Learning, évaluation et comparaison de modèles, interprétabilité, développement d'applications web avec Streamlit, conception d'API avec FastAPI, et gestion de projet.

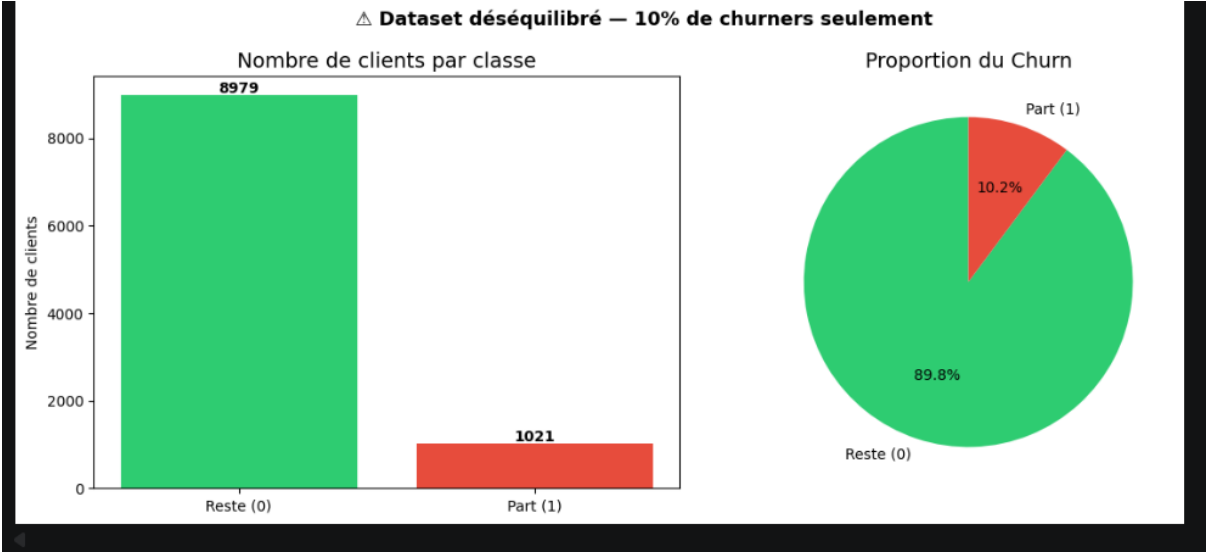
En synthèse : nous sommes passés d'un CSV de 10 000 lignes à un système intelligent prêt pour la production, capable d'identifier 85% des clients à risque et de quantifier 862 640 € de revenus menacés avec une interface décisionnelle et une API REST documentée.

Anexe 1 : diagramme de ganntt

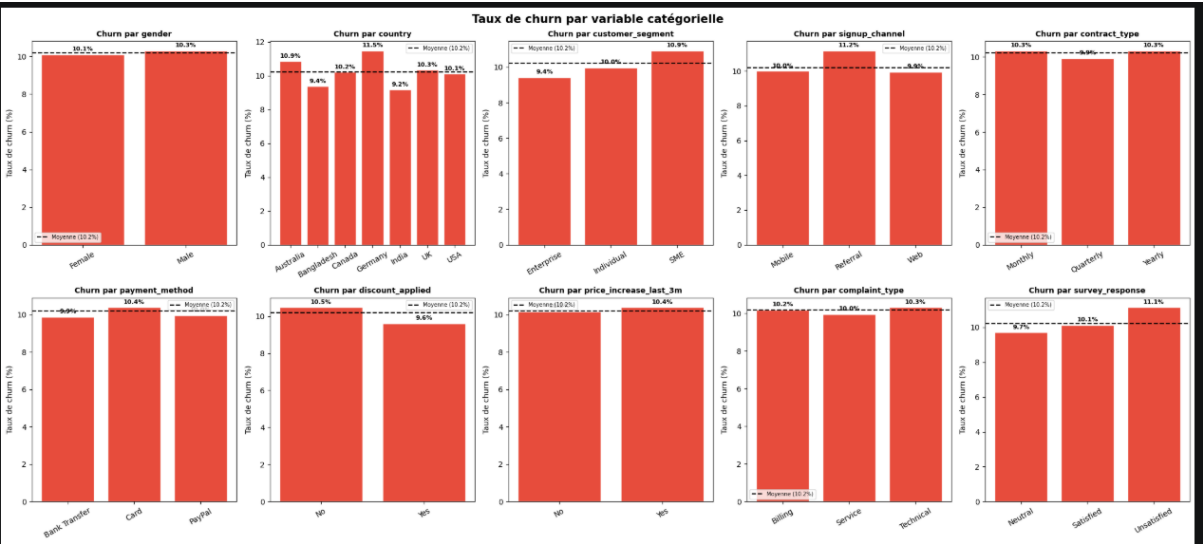
Phase / Livrable	jours 1	jours 2	jours 3	jours 4	jours 5	jours 6	jours 7	Livrable
EDA — Exploration des données								EDA.ipynb
Prétraitement & Pipeline								preprocessing.ipynb
Modélisation (4 modèles × 2 tâches)								model_*.ipynb
Comparaison finale & SHAP								comparaison_finale.ipynb
Dashboard Streamlit								app.py
API REST FastAPI								main.py
Rédaction rapport & soutenance								Rapport + slides

Annexe 2 — Visualisations EDA

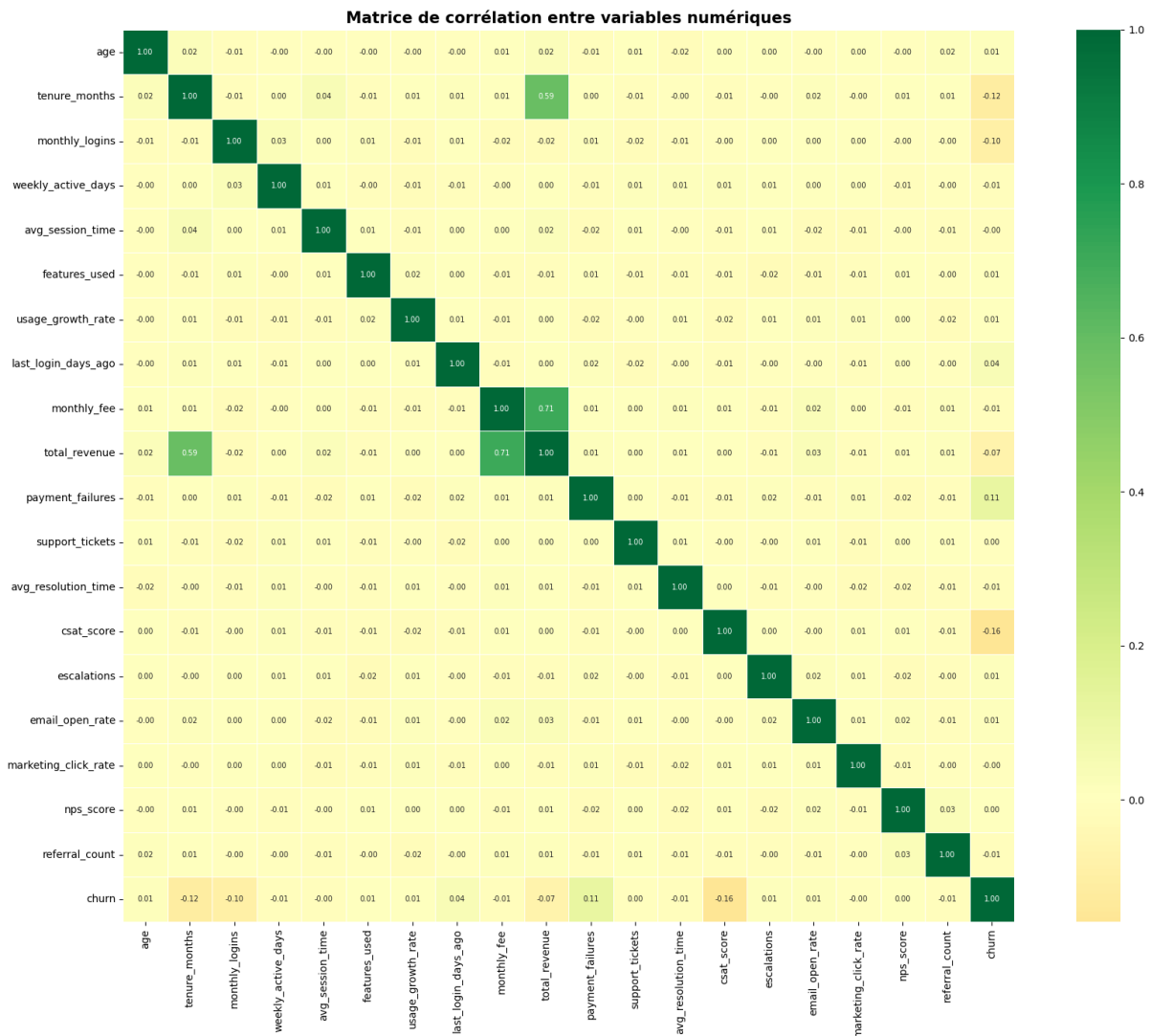
Distribution de la variable cible churn :



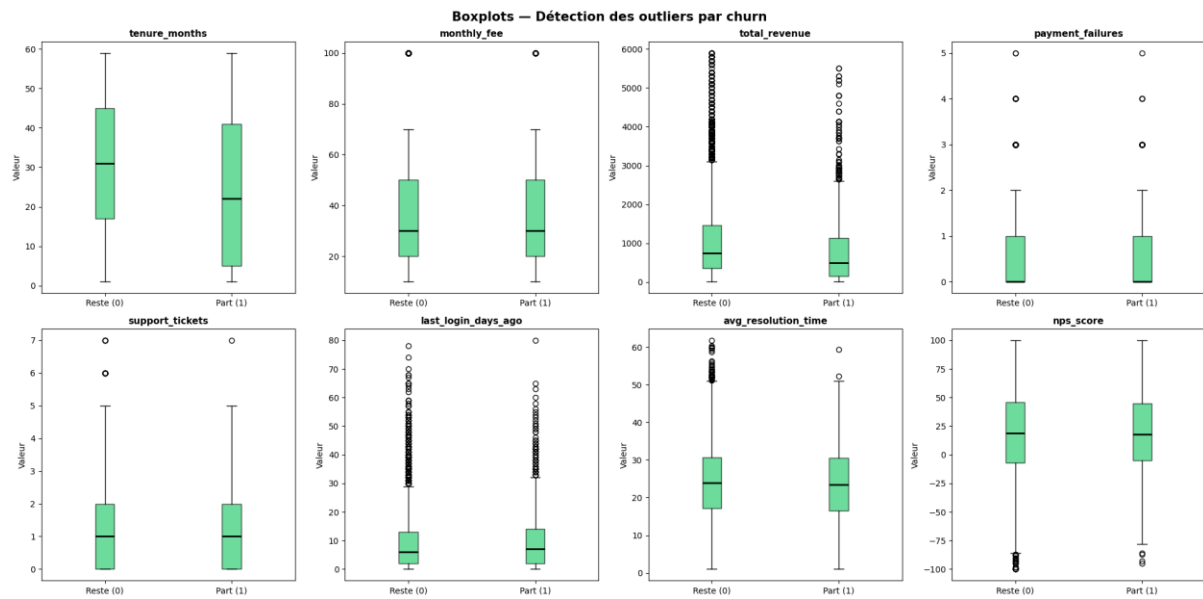
Taux de churn



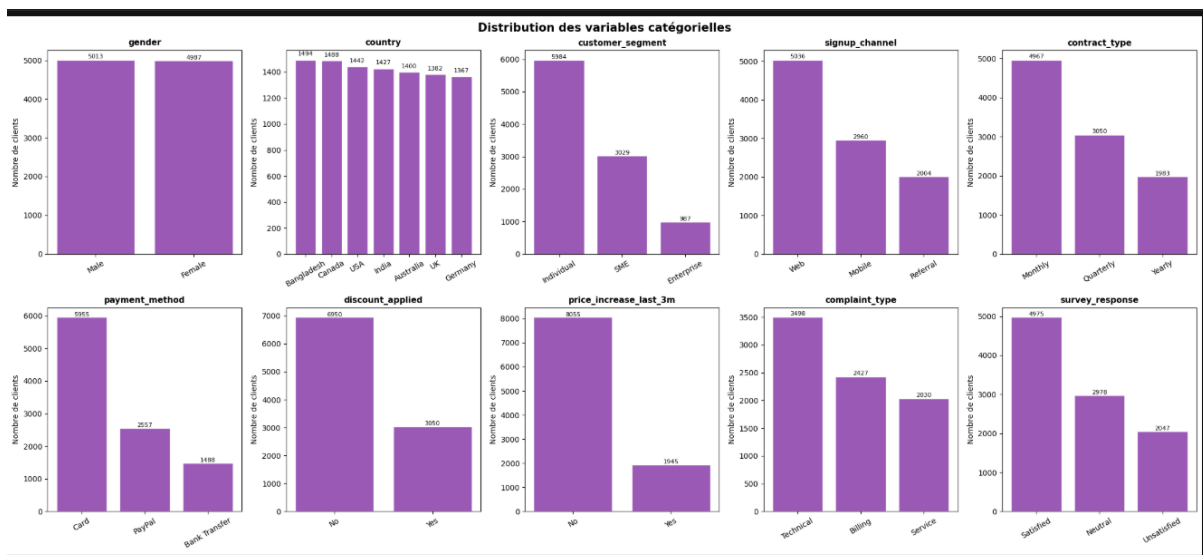
Heatmap de corrélation des variables numériques :



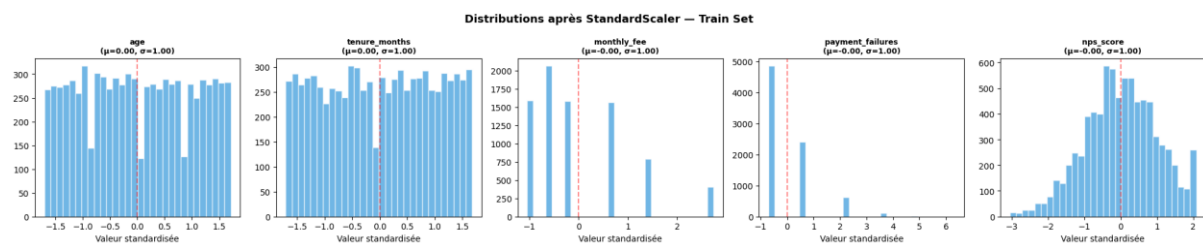
Boxplots des outliers (churners vs non-churners)



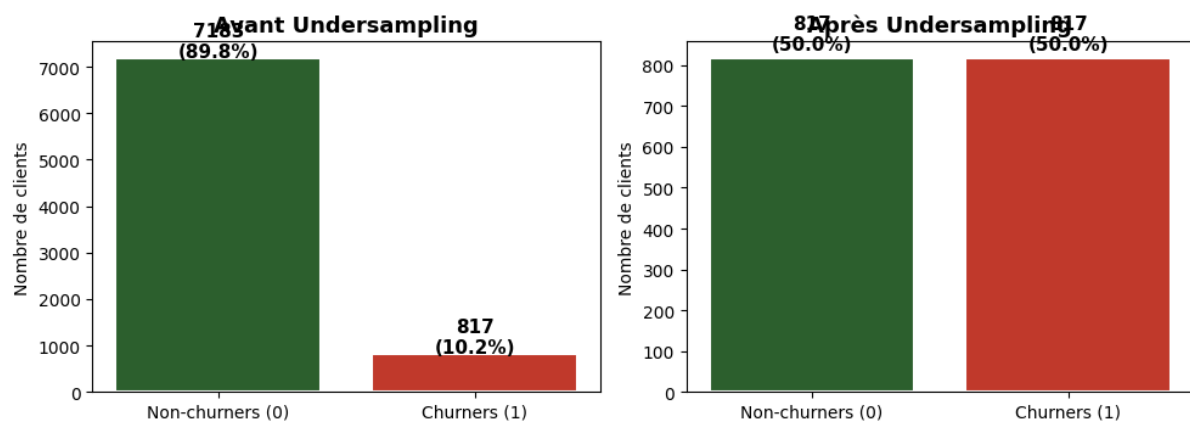
Distribution des variable catégorielles



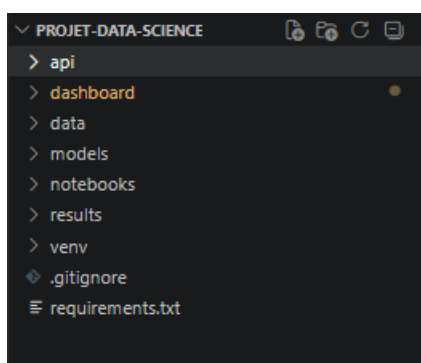
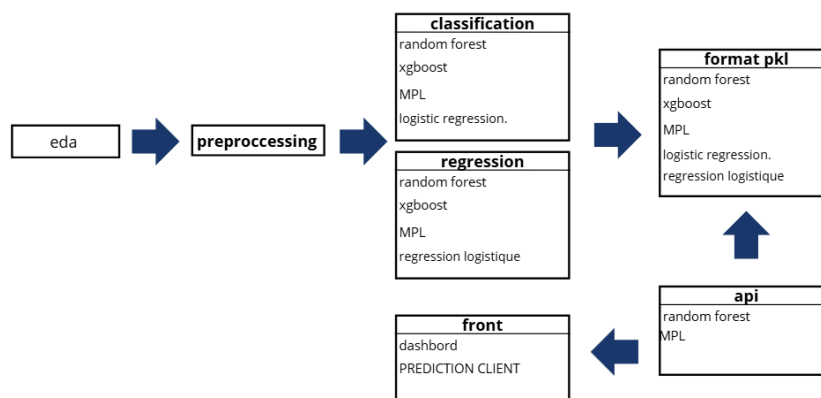
Annexe 3



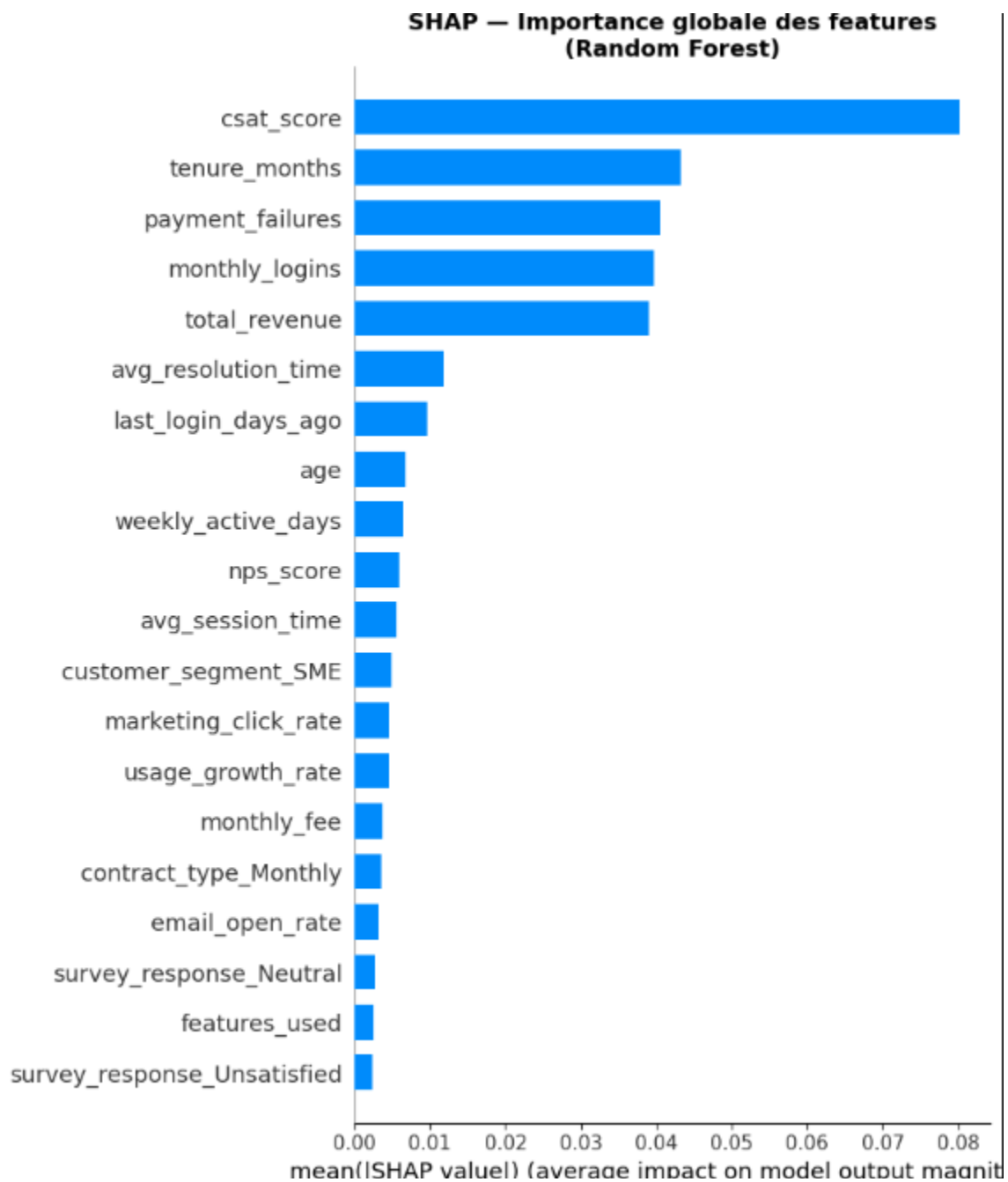
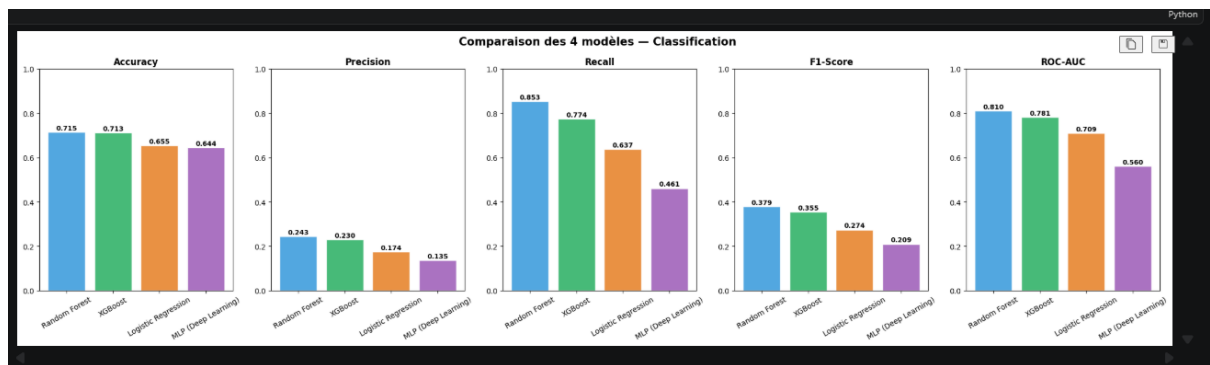
Équilibrage — Undersampling sur le Train Set uniquement

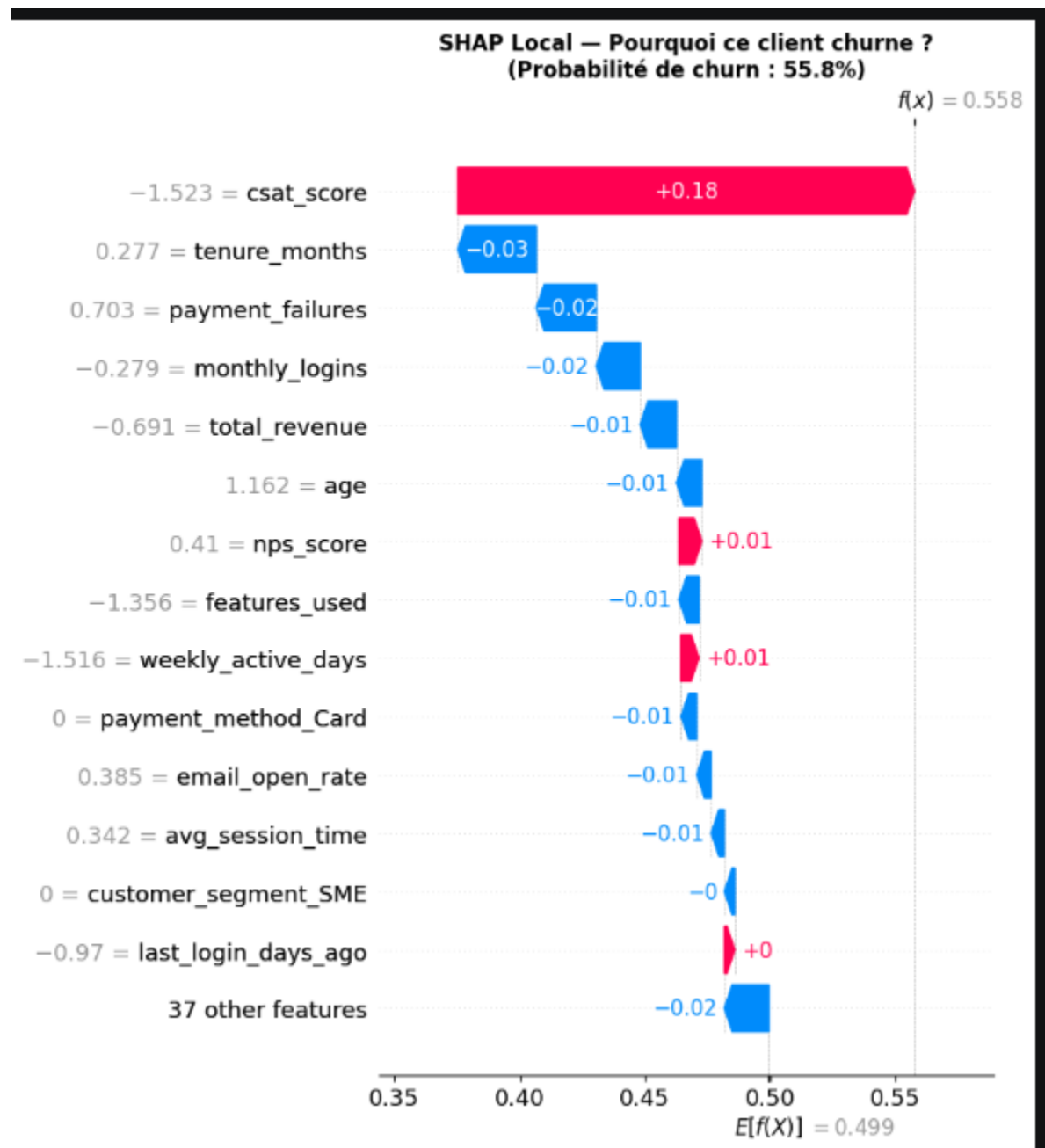


Annexe 4

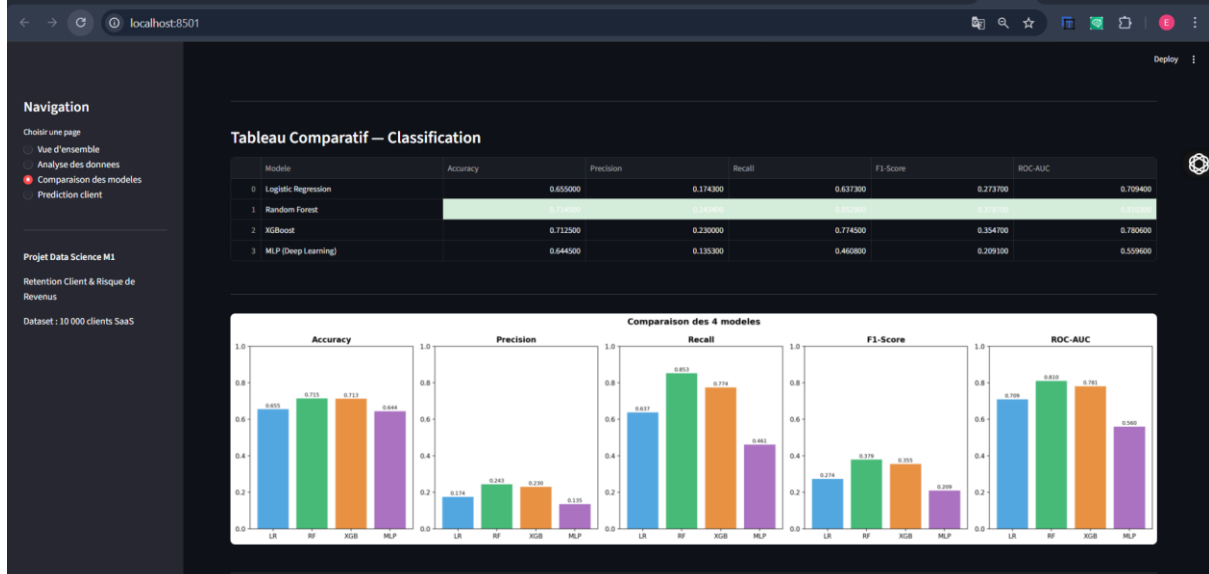
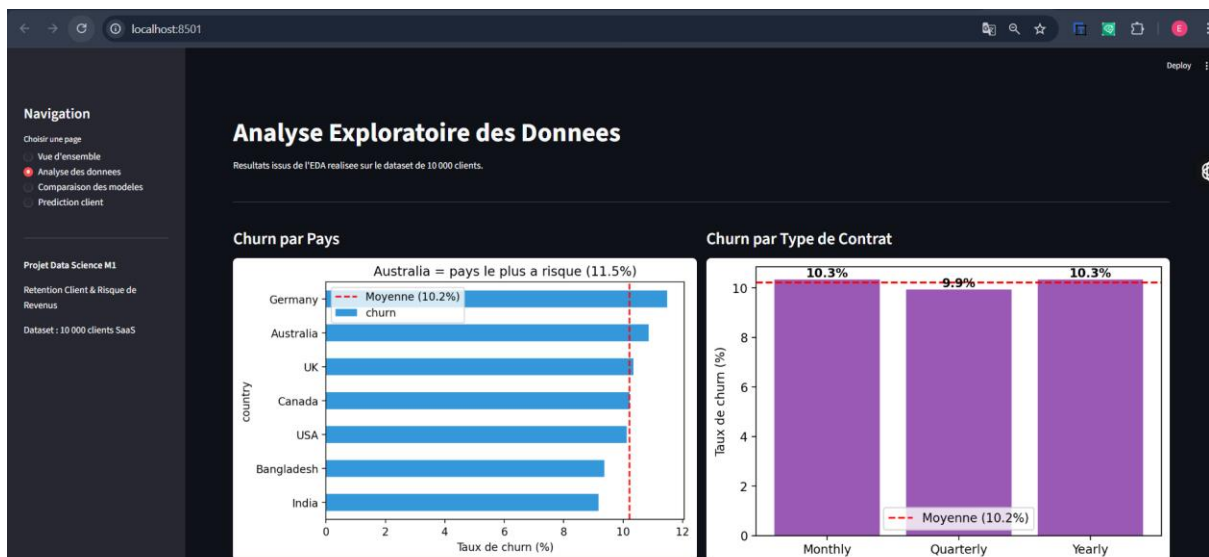
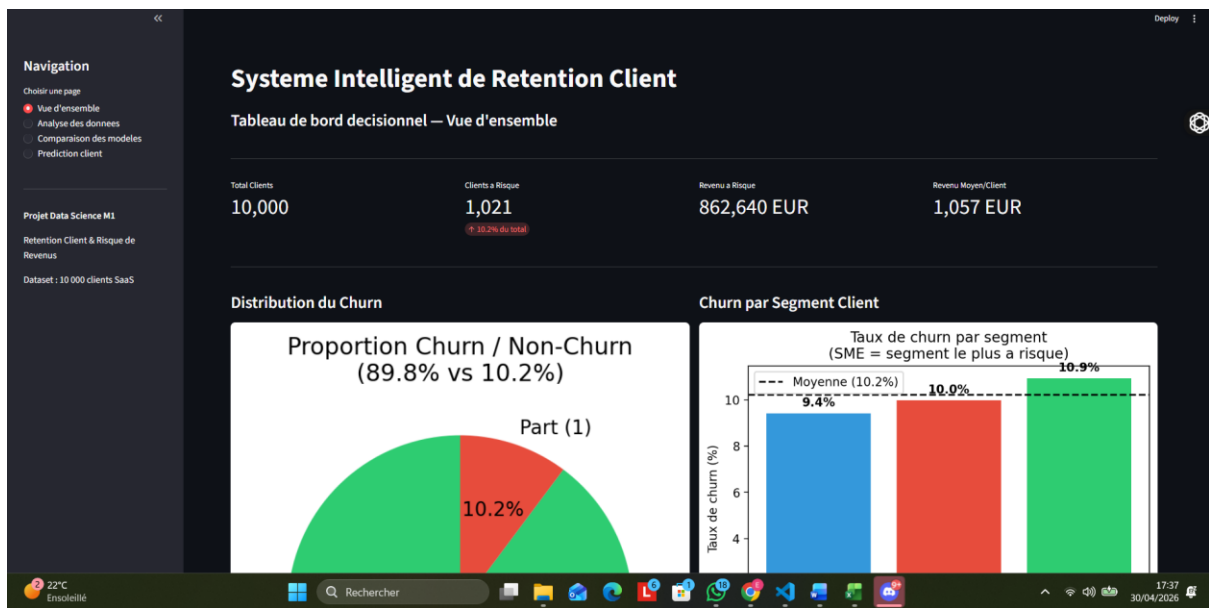


Annexe 5





Annexe 6



Navigation

Choisir une page

Vue d'ensemble

Analyse des données

Comparaison des modèles

Prediction client

Projet Data Science M1

Retention Client & Risque de Revenus

Dataset : 10 000 clients SaaS

Simulation — Prediction pour un Client

Entrez les caractéristiques d'un client pour obtenir sa probabilité de churn et son revenu à risque.

Les predictions sont generees par l'API FastAPI — Classification : Random Forest | Regression : MLP

Profil Client

Age

35

Genre

Male

Pays

Bangladesh

Segment

Individual

Anciennete (mois)

12

Canal

Web

Contrat

Utilisation

Connexions/mois

15

Jours actifs/semaine

3

Duree session (min)

15.00

Features utilisees

5

Croissance usage

0.00

Derniere connexion (jours)

5

Facturation & Support

Abonnement mensuel (EUR)

30

Revenu total (EUR)

500

Paiement

Card

Echecs paiement

0

Remise

No

Hausse prix recente

No

Tickets support

1

Annexe 7

localhost:8000/docs/#/

API Rétention Client

1.0.0

OAS 3.1

openapi.json

API REST pour la prédiction du churn client et l'estimation du revenu à risque.

Endpoints disponibles

- **GET /health** → Vérifie que l'API est active

- **POST /predict** → Prédit le churn et le revenu à risque

- **GET /model-info** → Informations sur les modèles

Santé

GET /health Health Check

Prédiction

POST /predict Predict

Informations

GET /model-info Model Info

Annex 8

